Desarrollo Web en Entorno Cliente

Control Flow: directivas y signals

Índice

Esquema 3

Material de estudio 4

6.1. Introducción y objetivos 4

6.2. Signals 4

6.3. Directivas de atributo 7

6.4. Directivas condicionales 11

6.5. Directivas condicionales múltiples (@switch) 13

6.6. Directivas de bucle 14

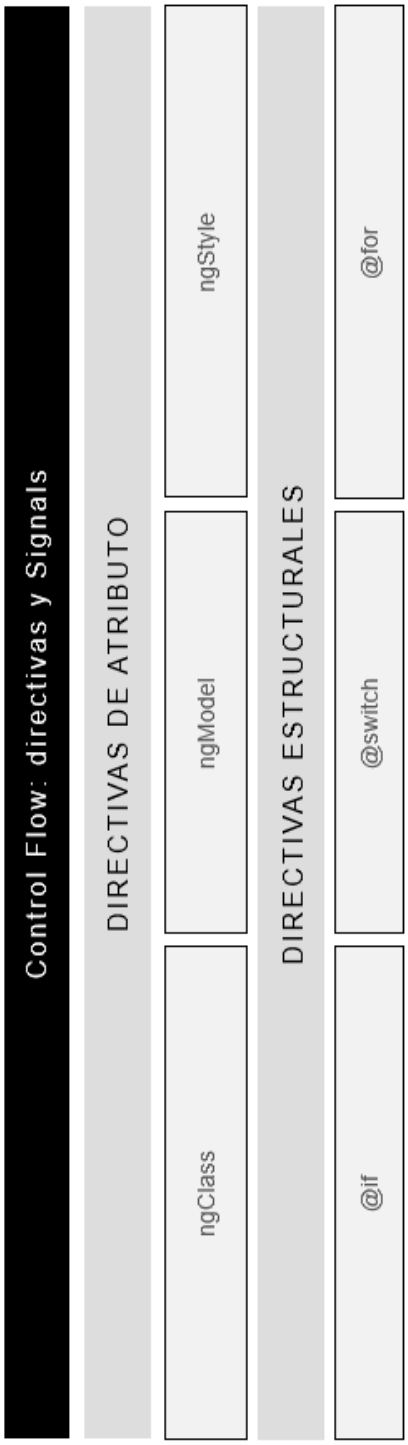
6.7. Creación de tu propia directiva 16

A fondo 19

Entrenamientos 20

Test 23

Esquema


Control Flow: directivas y Signals
DIRECTIVAS DE ATRIBUTO
ngClass
ngModel
ngStyle
DIRECTIVAS ESTRUCTURALES
@if
@switch
@for

Material de estudio

6.1. Introducción y objetivos

En este tema vamos a tratar el tema de las directivas y los signals. Podemos considerar como directivas, cualquiera de los elementos que representamos dentro de las plantillas de nuestras páginas web, ya sean etiquetas o atributos.

El objetivo de este tema es conocer los diferentes tipos de directivas y aprender a aplicarlos, para si obtener diferentes resultados dentro de nuestro HTML del componente.

Las Directivas extienden la funcionalidad del HTML usando para ello una nueva sintaxis. Con ella podemos usar lógica que será ejecutada en el DOM (Document Object Model) y así «olvidarnos» del manejo del DOM que hacía JavaScript de forma nativa, ayudándonos a gestionar la aparición, replica y estilos de nuestros elementos en Angular.

6.2. Signals

En Angular, los signals son una reciente adición que introducen un sistema de reactividad basado en valores, similar al concepto de "observables" pero con un enfoque más directo y sencillo. Los signals permiten trabajar con datos reactivos de manera más intuitiva, aprovechando el poder de las funciones reactivas sin necesidad de suscribirse y manejar múltiples operadores como ocurre con los observables. Actualmente aun se encuentran en developer preview en la versión de Angular 18.

Los signals representan una gran mejora para los desarrolladores de Angular, especialmente en aplicaciones donde se necesita un sistema de reactividad simple y eficiente. Son ideales para gestionar estados locales de componentes y son una opción excelente para complementar el uso de observables en Angular.

¿Qué son los signals?

Un signal en Angular representa un valor reactivo que puede cambiar a lo largo del tiempo y al que otros componentes o funciones pueden reaccionar automáticamente cuando ese valor cambia. Los signals permiten el rastreo y la actualización automática de valores en la interfaz sin necesidad de realizar seguimiento manual de sus dependencias.

Los signals tienen las siguientes propiedades:

- Mutable e Inmutable: Se puede crear un signal que sea modificable (mutable) o que mantenga un estado fijo una vez inicializado (inmutable).
- Lectura de valor sencilla: Para acceder al valor de un signal, simplemente se llama al signal como una función.
- Automático en el cambio de estado: Cuando el valor de un signal cambia, todos los valores o funciones dependientes se actualizan de forma automática.
- Composición declarativa: Al definir signals y funciones dependientes, Angular puede determinar automáticamente las dependencias, haciendo que la programación declarativa sea sencilla.
- Integración con la vista: Los signals están diseñados para integrarse fácilmente con los templates de Angular, simplificando el proceso de enlazar datos reactivamente en la interfaz.

Ejemplo sencillo de declaración de signal:

```
import { signal } from '@angular/core';

// Definimos un signal de contador
const contador = signal(0);

// Leer el valor
console.log(contador()); // Imprime 0
```

```
// Actualizar el valor
contador.set(5);
console.log(contador()); // Imprime 5
```

Input Signals

Un Input Signal en Angular es una propiedad que permite a un componente recibir valores de entrada en tiempo real como un signal. Esta funcionalidad es especialmente útil en aplicaciones con datos dinámicos, ya que el componente reacciona de inmediato a cualquier cambio en los valores de entrada sin necesidad de implementar manualmente la reactividad.

Los Input signals representan una evolución de las tradicionales propiedades @Input de un componente, pero con capacidades reactivas que permiten a los componentes recibir datos de entrada en forma de signals. Esto hace que los componentes respondan automáticamente a los cambios en los valores de entrada, lo que simplifica el flujo de datos reactivos dentro de la aplicación. Podemos destacar las siguientes bondades o propiedades:

- Reactividad Automática: Los Input Signals reaccionan automáticamente cuando su valor cambia, actualizando la vista sin necesidad de suscripción.
- Mayor rendimiento en la detección de cambios: Angular optimiza los cambios en los Input Signals, evitando recalcular toda la vista y centrándose solo en los cambios necesarios.
- Sin necesidad de @Input tradicional: A diferencia de las propiedades @Input, los Input Signals ya están optimizados para reactividad y actualizaciones automáticas.
- Propiedades de entrada reactivas: Los Input Signals permiten que las propiedades de entrada actúen como signals, lo que habilita un flujo de datos en tiempo real entre componentes.
- Declarativo: Al igual que otros signals, se definen de manera declarativa, lo que hace que el código sea más intuitivo y fácil de mantener.

Angular soporta dos variantes de inputs:

- Optional inputs: Los inputs son opcionales por defecto, a menos que uses input.required. Puedes especificar un valor inicial explícito, o Angular usará undefined implícitamente.
- Requiered inputs: Las entradas requeridas siempre tienen un valor del tipo de entrada dado. Se declaran usando la función input.required.

Un *input* es reconocida automáticamente por Angular siempre que utilices las funciones *input* o *input.required* como inicializador de miembros de la clase.

```
import {Component, input} from '@angular/core';
@Component({...})
export class MyComp {
  // optional
  firstName = input<string>();         // InputSignal<string|undefined>
  age = input(0);                      // InputSignal<number>
  // required
  lastName = input.required<string>(); // InputSignal<string>
}
```

6.3. Directivas de atributo

Podemos considerar como directivas, cualquiera de los elementos que representamos dentro de las plantillas de nuestras páginas web, ya sean etiquetas o atributos.

Por lo tanto, los componentes que definamos en nuestras aplicaciones también pueden ser considerados como directivas. En este caso, lo destacable es que estas directivas especiales llevan asociadas una plantilla HTML.

Aparte, disponemos de dos tipos diferentes de directivas: de atributo y estructurales. Las directivas de atributo se encargan de modificar la apariencia o el comportamiento de cualquiera de los elementos que podamos encontrar dentro del DOM.

Existen tres, una de ellas ya la conocemos que es la que nos permite hacer el two Data Binding.

- ngClass: nos permite añadir dinámicamente clases de css de forma fácil. Tiene varias formas de implementarse y me da mucha más versatilidad que el property biding.
- ngModel: Necesario para aplicar el Two Data Biding, asocia el valor de una propiedad del TS a un elemento en el que el usuario puede introducir valores, como por ejemplo un input.
- ngStyle: permite asignar varios estilos en línea a un elemento.

ngStyle

Mediante esta directiva podemos aplicar a nuestras plantillas, un objeto con los diferentes estilos que necesitemos en cada caso.

Los valores asignados a cada uno de los estilos que apliquemos a través de esta directiva pueden ser fijos o, por el contrario, podemos utilizar alguna de las variables de nuestro componente para darle un carácter más dinámico.

Lo vemos con una serie de ejemplos. Todo el código que vais a visualizar está en su mayoría creado dentro del HTML del componente en el que trabajéis.

```
<p [ngStyle]="{'color': 'blue', 'font-size': '12px', 'font-weight':
'bold'}">Párrafo de texto</p>
```

También podemos asignar los valores a través de las propiedades o métodos de nuestra clase

```
import { Component, OnInit, SimpleChange } from '@angular/core';
@Component({
  selector: 'hola',
  template: `<p [ngStyle]="paragStyles">Párrafo de texto</p>`
```

```
})
export class HolaComponent {
  weight = 'lighter'
  paragStyles = {
    'color': this.getRandomColor(),
    'font-size': '14px',
    'weight': this.weight
  }
  getRandomColor() {
    return '#' + Math.floor(Math.random() * 16777215).toString(16)
  }
}
```

Y el resultado en esta ocasión en el HTML se visualizaría de esta forma

```
<p [ngStyle]="{'color': 'blue', 'font-size': '12px', 'font-weight':
weight}">Párrafo de texto</p>

<p [ngStyle]="paragStyles">Párrafo de texto</p>
```

ngClass

Mediante la directiva ngClass podemos asignar clases CSS a nuestros componentes. Podemos hacerlo de tres maneras diferentes.

1. Mediante un array de clases:

Asignamos a la directiva un array con todas las clases de CSS que queremos aplicarle, por supuesto para que este funcione dichas clases tiene que estar creadas dentro del CSS del componente.

```
<p class="fondoAzul textoRojo" [ngClass]="['textoRojo',
'fondoAzul']">Lorem aliquid!</p>
```

2. Mediante un string:

Esta es la forma menos recomendable porque es muy parecida al usar una clase estática de las de CSS de toda la vida, pero es una alternativa que puede ser

interesante en algún caso, como por ejemplo recibir todo un string como propiedad de una clase.

<p [ngClass]="'texto-negrita verde'">Párrafo de texto</p>

3. Mediante un objeto:

Mediante un objeto, en el cual se especifica mediante un boolean, qué clases se aplican y cuáles no.

Esta quizás es la más interesante ya que te permite activar o desactivar valores dependientes de valores de propiedades que tenga tu TypeScript.

```
<p [ngClass]="{'fondoRojo': activo, 'fondoAzul': !activo}">Lorem  eaque!</p>
<button (click)="cambiarActivo()">Cambiar activo</button>
```

Si creásemos una propiedad dentro del nuestra clase de componente que se llamase activo la clase de CSS fondoRojo y fondoAzul se activarían siempre y cuando su valor fuese true o false.

El código en nuestro componente quedaría de la siguiente manera.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-ng-class',
  templateUrl: './ng-class.component.html',
  styleUrls: ['./ng-class.component.css']
})
export class NgClassComponent implements OnInit {

  texto: string = "textoRojo"
  activo: boolean = false
  constructor() { }

  ngOnInit(): void {
```

```
  }

  cambiarEstilo() {
    this.texto = (this.texto == 'textoRojo') ? 'textoVerde' : 'textoRojo'
  }

  cambiarActivo() {
    this.activo = !this.activo;
  }
}
```

6.4. Directivas condicionales

En Angular, a partir de la versión 17, se han introducido mejoras significativas en el control flow o flujo de control para la lógica de plantillas, con especial atención a las directivas *ngIf, *ngFor y *ngSwitch. Aunque estas directivas no se han reemplazado por @if, @for o @switch, las nuevas optimizaciones y capacidades permiten un manejo mucho más potente y eficiente de las estructuras condicionales y de bucles en las vistas.

@if que nos permite cargar o eliminar cierta parte de la estructura de un componente a través de una expresión booleana. El bloque @if muestra condicionalmente su contenido cuando su expresión de condición es verdadera. Al contrario que las propiedades de css display o visibility que tan solo lo ocultan de la vista, pero lo cargan dentro del componente.

```
@if (a > b) {
  <p>{{a}} is greater than {{b}}</p>
}
```

Si se desea mostrar contenido alternativo, se puede hacer proporcionando cualquier número de bloques @else if y un bloque @else singular.

```
@if (a > b) {
  {{a}} is greater than {{b}}
```

```
} @else if (b > a) {
  {{a}} is less than {{b}}
} @else {
  {{a}} is equal to {{b}}
}
```

El condicional @if permite guardar el resultado de la expresión condicional en una variable para reutilizarlo dentro del bloque. Esto puede ser útil para referenciar expresiones más largas que serían más fáciles de leer y mantener dentro de la plantilla.

```
@if (user.profile.settings.startDate; as startDate) {
  {{ startDate }}
}
```

¿Por qué eliminamos el contenido en vez de ocultarlo?

Podríamos realizar el mismo trabajo que hace esta directiva con la aplicación de ciertos estilos CSS que nos permitan mostrar/ocultar el HTML.

El comportamiento con CSS es el siguiente:

- Cuando ocultamos un componente o parte de nuestra página, su comportamiento sigue activo.
- Sigue escuchando los posibles eventos que reciba la aplicación.
- Angular seguirá escuchando los posibles cambios que le afecten y le seguirá enviando los datos que se modifiquen.
- Sigue usando recursos de nuestra aplicación.
- Por otro lado, el cambio de oculto a no es muy rápido.

El trabajo con @if es diferente:

- Si ocultamos bloques de código, afecta a su rendimiento, ya que se elimina del DOM.
- Deja de recibir evento.

- El componente es destruido y se ejecutan los métodos finales del ciclo de vida del componente.
- En este caso, se emplean muchos recursos para regenerar el componente.

6.5. Directivas condicionales múltiples (@switch)

Aunque el bloque @if es ideal para la mayoría de los casos, el bloque @switch proporciona una sintaxis alternativa para mostrar datos de forma condicional. Su sintaxis se parece mucho a la sentencia switch de JavaScript.

```
@switch (userPermissions) {
  @case ('admin') {
    <app-admin-dashboard />
  }
  @case ('reviewer') {
    <app-reviewer-dashboard />
  }
  @case ('editor') {
    <app-editor-dashboard />
  }
  @default {
    <app-viewer-dashboard />
  }
}
```

El valor de la expresión condicional se compara con la expresión case mediante el operador triple igual (===). @switch no tiene fallthrough, por lo que no necesita un equivalente a una sentencia break o return en el bloque.

Opcionalmente puede incluir un bloque @default. El contenido del bloque @default se muestra si ninguna de las expresiones case precedentes coincide con el valor de switch. Si ningún @case coincide con la expresión y no hay bloque @default, no se muestra nada.

6.6. Directivas de bucle

El bloque @for recorre una colección y renderiza repetidamente el contenido de un bloque. La colección puede ser cualquier iterable de JavaScript, pero Angular tiene optimizaciones de rendimiento adicionales para los valores Array.

Un bucle @for típico tiene este aspecto:

```
@for (item of items; track item.id) {
  {{ item.name }}
}
```

¿Por qué es importante la pista en los bloques @for?

La expresión *track* permite a Angular mantener una relación entre tus datos y los nodos DOM de la página. Esto permite a Angular optimizar el rendimiento mediante la ejecución de las operaciones DOM mínimas necesarias cuando cambian los datos.

El uso eficaz de la expresión *track* puede mejorar significativamente el rendimiento de renderización de la aplicación al realizar bucles sobre colecciones de datos.

Se debe de seleccionar una propiedad que identifique de forma exclusiva cada elemento en la expresión de seguimiento. Si nuestro modelo de datos incluye una propiedad de identificación única, comúnmente id o uuid, utilizaremos este valor. Si nuestros datos no incluyen un campo como este, se debe considerar añadir uno.

Para colecciones estáticas que nunca cambian, puedes usar $index para decirle a Angular que rastree cada elemento por su índice en la colección. Si no hay otra opción disponible, puedes especificar *identity*. Esto le dice a Angular que rastree el elemento por su identidad de referencia usando el operador triple-igual (===). Es recomendable evitat esta opción siempre que sea posible, ya que puede llevar a actualizaciones de renderizado significativamente más lentas, ya que Angular no tiene forma de asignar qué elemento de datos corresponde a qué nodos DOM.

Dentro de los bloques @for, siempre hay disponibles varias variables implícitas:

- $count: Número de elementos de una colección iterada.
- $index: Número de elementos de una colección iterada.
- $first: Si la fila actual es la primera fila.
- $last: Si la fila actual es la última fila.
- $even: Si el índice de la fila actual es par.
- $odd: Whether the current row index is odd.

Estas variables están siempre disponibles con estos nombres, pero se pueden aliasear mediante una declaración let:

```
@for (item of items; track item.id; let idx = $index, e = $even) {
  <p>Item #{{ idx }}: {{ item.name }}</p>
}
```

Bloques @for con el bloque @empty

Se puede incluir opcionalmente una sección @empty inmediatamente después del contenido del bloque @for. El contenido del bloque @vacío se muestra cuando no hay elementos:

```
@for (item of items; track item.name) {
  <li> {{ item.name }}</li>
} @empty {
  <li aria-hidden="true"> There are no items. </li>
}
```

6.7. Creación de tu propia directiva

Una vez conocemos las diferentes directivas que podemos utilizar dentro de nuestras plantillas, vamos a analizar cómo podemos crear las nuestras propias.

Para generar nuestras propias directivas tenemos a nuestra disposición el decorador @Directive.

Para crear una directiva usamos este código desde terminal al igual que hacíamos para crear componentes.

```
ng g directive nomDirectiva
```

Este decorador se aplica sobre una clase como las vistas anteriormente.

```
import { Directive } from '@angular/core';
@Directive({
  selector: "[neoColor]"
})
export class ColorDirective {
}
```

El primer elemento a destacar es el atributo selector, mediante el cual indicamos cómo vamos a representar nuestra directiva en el momento de utilizarla.

Para poder definir un selector en nuestras directivas necesitamos usar nomenclatura CSS.

Si mantenemos la sintaxis vista en el ejemplo anterior, el modo para ejecutar dicha directiva sería el siguiente:

```
<p neoColor>Contenido</p>
```

Aunque si lo preferimos, podemos definir nuestra directiva como una clase más de CSS aplicada sobre cualquiera de los elementos de nuestro DOM.

Para alcanzar este objetivo tendremos que modificar el selector y la forma de aplicarlo:

```
@Directive({
  selector: ".neoColor"
})
```

```
<p class="neoColor">Contenido</p>
```

Como cualquiera de las clases que desarrollamos dentro de nuestra estructura de aplicación Angular, la clase que representa nuestra directiva dispone de su método constructor.

Dentro del método constructor podemos inyectar un objeto de tipo ElementRef, el cual nos da acceso al elemento del DOM sobre el cual estamos aplicando dicha directiva.

Para poder interactuar con dicho elemento tenemos que hacerlo a través de la propiedad nativeElement del objeto inyectado.

Vemos en un ejemplo cómo podríamos modificar algún estilo del elemento sobre el cual aplicamos la directiva.

```
import { Directive, ElementRef } from '@angular/core';
@Directive({
  selector: "[neoColor]"
})
export class ColorDirective {
  constructor(el: ElementRef) {
    el.nativeElement.style.color = 'red'
  }
```

```
}
```

En el ejemplo anterior estamos asumiendo que nuestra aplicación se está ejecutando sobre un navegador, pero esto no siempre será así.

Angular se ha diseñado para que pueda ser utilizado en diferentes entornos, incluso podríamos trabajar del lado servidor gracias a Node o en cualquier dispositivo móvil.

Por lo que disponemos de una herramienta mediante la cual podemos modificar propiedades de nuestros elementos, independientemente de la plataforma en la que nos encontremos, se trata del objeto Renderer.

Modificamos el ejemplo anterior:

```
import { Directive, ElementRef, Renderer } from '@angular/core';
@Directive({
  selector: "[neoColor]"
})
export class ColorDirective {
  constructor(el: ElementRef, renderer: Renderer) {
    renderer.setElementStyle(el.nativeElement, 'color', 'red')
  }

}
```

En este caso, si visualizamos nuestra aplicación en un entorno diferente al web, el objeto Renderer será el encargado de llamar a las funciones apropiadas en cada caso para conseguir la modificación que estamos aplicando.

De esta manera eliminamos la limitación del navegador.